

ClamshellRS

A Game Boy Advance SP Emulator in Rust

CPU · Memory Bus · Scheduler · PPU · Audio · Cartridge

A Component-by-Component Build Plan — 108 working days across nineteen self-contained subsystems

Target hardware	AGB-001 / AGS-001 / AGS-101 — identical internals
Processor	ARM7TDMI, ARMv4T, 16.777216 MHz, dual instruction set
Display	240 x 160, six display modes, ~59.73 Hz
Language	Rust (stable), no unsafe required in the core
Duration	108 working days at 4 hrs/day — roughly 22 weeks
Structure	Nineteen components, each built, tested and closed out in turn

How to Use This Plan

This plan is organised by **component** rather than by feature or by week. Each component is one subsystem of the machine: you build it, you test it, you close it out, and then you move to the next one. The reason for this shape is that an emulator has an unusually rigid dependency chain. In most software projects you can work on two things at once and integrate later. Here you cannot: nothing above the processor can be meaningfully validated until the processor is correct, and nothing above the memory bus can run at all.

That rigidity is also the plan's main structural risk, and it is worth naming on page one. Components 2 and 3 — the CPU — take twenty working days and produce nothing you can look at. This is the single most common place people abandon an emulator project. The mitigation is built into the ordering: the verification harness in Component 4 gets pulled as early as it can usefully go, so that instead of twenty days of silence you get a steadily advancing count of passing tests.

Pacing. Roughly 108 working days at 4 hrs/day, 5 days/week, which is about 22 weeks or 5 to 5.5 calendar months with normal interruptions. Buffer days are built into every component and they are real slack, not days to work ahead — several components (the CPU, EEPROM, audio synchronisation) reliably run long for reasons that are not your fault. Someone who has written an emulator before can compress this considerably; someone meeting ARM assembly for the first time should not plan to.

What makes this project different from an application build. You are not designing the requirements. The hardware is the specification, it is comprehensively documented, and it is not negotiable — if your output differs from the real machine, you are wrong, full stop. Every creative decision you make is therefore architectural rather than behavioural: how to model ownership in Rust, how to schedule events, where to sit on the accuracy-versus-speed curve, what to render per scanline. Those decisions are the portfolio value here, and the plan asks you to write each of them down as you make it.

Use AI assistance the same way you would on any build: heavily for boilerplate (instruction dispatch scaffolding, register bit-field definitions, windowing and audio device setup, test harness plumbing) and for explaining unfamiliar ARM semantics. Keep ownership of the decisions listed above, and keep ownership of your honest read on what your emulator does and does not get right — a documented scope boundary reads as judgment, an undocumented one reads as an oversight.

A Critical Framing Point: What You Are Actually Building

Read this before you start.

1. The SP is not a different machine. The AGS-001 and AGS-101 contain the same ARM7TDMI, the same memory map, the same picture processing unit and the same audio hardware as the original AGB-001. The differences are a clamshell shell, a rechargeable battery, a frontlit or backlit screen, and a missing headphone jack. There is no separate SP core to write. You are building a Game Boy Advance emulator. SP-ness appears in exactly one place in this entire plan — Component 16, where you decide what the panel looked like — and that is a presentation-layer decision, not a hardware one. Everything before it is shared with every other GBA model.

2. Legality has a specific shape you need to understand up front. Writing an emulator is lawful. The 16 KB GBA BIOS is copyrighted Nintendo code and cannot be distributed with your project; users supply their own dump, or you ship a clean-room open-source replacement, which can legally be bundled. Commercial cartridge dumps are likewise not yours to distribute or to commit to a repository. Your automated test suite must therefore be built entirely from homebrew and public test ROMs — which is fortunate, because those are excellent and are the backbone of Component 4. Decide your BIOS strategy on Day 3 and do not defer it.

3. 'It runs games' and 'it is accurate' are two different projects. Decide which one you are building before you start. A compatibility-focused emulator gets the overwhelming majority of commercial games playable by roughly Component 13. An accuracy-focused one spends its final third chasing a long tail of behaviours that a handful of games and a great many demos depend on. Both are entirely legitimate. What is not legitimate is claiming the second while having built the first: a reviewer who knows the domain will ask you about the cartridge prefetch buffer, and 'I chose not to implement it, and here is the scope reasoning' is a far stronger answer than a vague one.

Before You Start: What You'll Need

A shorter list than most projects — no cloud accounts, no API keys, no service provisioning. The friction in this build is entirely intellectual, and it starts on Day 9.

Item	Why	Notes
Rust toolchain, stable	The entire project	Via rustup, plus clippy and rustfmt. No nightly features are required anywhere in this plan.
A profiler	Component 17, and sanity checks throughout	cargo-flamegraph or samplify. Install it now so it is not a Day 100 detour.
A GBA BIOS dump, or an open replacement	Boot sequence and system calls	You must supply your own dump. A clean-room open-source replacement exists and can be legally bundled — plan to support both.
A reference emulator	Trace diffing, the most valuable debugging technique in this project	mGBA is the recommended choice: accurate, well documented, and its authors have written extensively about the hardware.
Public test ROM suites	Your entire correctness story	jsmolka's gba-tests, ARMWrestler, FuzzARM, the mGBA test suite, the hades PPU tests, and the AGS aging cartridge.
GBATEK, community-maintained fork	The register-level hardware reference	Keep it open permanently. Every number in this document traces back to it. The original edition has known errata; use the maintained fork.
ARM7TDMI technical reference manual	Exact instruction semantics	Free from ARM. Pair it with the ARMv4 architecture reference manual for the edge cases.
TONC	What games actually do with the hardware	A GBA programming tutorial. Teaches intent rather than registers, which matters when you are trying to work out why a game is doing something odd.
ARM cross-toolchain binutils	Validating your disassembler	arm-none-eabi-objdump, for cross-checking Component 4's disassembler against a known-good implementation.
Legally obtained cartridge dumps	Real-world testing	Games you own. Never commit these to the repository.
GitHub account	Source control and CI	The test ROM suite runs in CI from Component 4 onward. You likely have this already.

Project Naming

ClamshellRS is used throughout this document. It reads as a deliberate reference to the SP's defining physical feature without being cute about it, and the suffix makes the language obvious at a glance — which matters more than it sounds when the project is a line item on a resume. Alternatives worth considering:

- **ClamshellRS** (recommended) — specific to the SP, unambiguous about the language
- **Frontlight** — references the AGS-001's defining feature; reads well, slightly obscure
- **agb-rs** — plain, conventional, immediately legible to anyone in the emulation community
- **Sable** — short and memorable, but says nothing about what it is

Table of Contents

How to Use This Plan	2
A Critical Framing Point: What You Are Actually Building	3
Before You Start: What You'll Need	4
Project Naming	4
Component 0 — Environment, Repo Foundation & Architectural Commitments	9
Day 1 — Toolchain and workspace skeleton	9
Day 2 — The ownership decision	9
Day 3 — Accuracy tier, CI, and the BIOS strategy	9
Component 1 — Memory Bus & Cartridge	11
Day 4 — The address map and region decode	11
Day 5 — Mirroring, and the one case that is not simple	11
Day 6 — BIOS read protection and open bus	12
Day 7 — Cartridge loading and header parsing	12
Day 8 — Access timing table and buffer	12
Component 2 — CPU Part One: Core Model and the ARM Instruction Set	13
Day 9 — Register file and mode banking	13
Day 10 — The pipeline model	13
Day 11 — Decode strategy	13
Day 12 — Condition codes and the barrel shifter	14
Days 13–14 — Data processing and status register transfer	14
Days 15–16 — Load and store, single and halfword	14
Days 17–18 — Block transfer, multiply, swap	15
Day 19 — Software interrupt and undefined instructions	15
Day 20 — Buffer and first partial test run	15
Component 3 — CPU Part Two: THUMB, Exceptions, and Getting to Green	17
Days 21–23 — The THUMB instruction formats	17
Day 24 — The exception mechanism	17
Days 25–27 — Getting the test suites to green	18
Day 28 — Buffer, and a hard gate	19
Component 4 — Verification Harness and Debug Tooling	20
Day 29 — Headless runner and the ROM test harness	20

Day 30 — Trace diffing	20
Day 31 — Disassembler	20
Day 32 — Save states	20
Day 33 — Debugger skeleton and buffer	21
Component 5 — Timing and the Scheduler	22
Day 34 — The timing model	22
Day 35 — Scheduler implementation	22
Day 36 — The main loop	22
Day 37 — Buffer and accuracy-tier check	23
Component 6 — Interrupts and Timers	24
Day 38 — The interrupt controller	24
Day 39 — Timers, scheduled	24
Day 40 — Timer edge cases	24
Day 41 — Buffer	25
Component 7 — DMA	26
Day 42 — Channel model and per-channel differences	26
Day 43 — Transfer execution and priority	26
Day 44 — Timing triggers and the special modes	26
Day 45 — Buffer and trigger validation	27
Component 8 — PPU Part One: Display Timing and Bitmap Modes	28
Day 46 — The rendering architecture decision	28
Day 47 — The video state machine	28
Day 48 — Colour and the framebuffer	28
Day 49 — The three bitmap modes	29
Day 50 — Buffer and first visible output	29
Component 9 — PPU Part Two: Tiled Backgrounds	30
Day 51 — Background control and tile addressing	30
Days 52–53 — Text background rendering	30
Days 54–55 — Affine backgrounds	30
Day 56 — Buffer and first title screens	31
Component 10 — PPU Part Three: Sprites	32
Day 57 — Object memory and the attribute layout	32
Days 58–59 — Sprite rendering	32

Day 60 — Affine sprites	32
Day 61 — Sprite video RAM restrictions and edge cases	33
Day 62 — Buffer and the sprite viewer	33
Component 11 — PPU Part Four: Windows, Blending, and Mosaic	34
Days 63–64 — Windows	34
Days 65–66 — Blending	34
Day 67 — Composition	34
Day 68 — Mosaic	35
Day 69 — Buffer and picture-unit validation sweep	35
Component 12 — Input	36
Day 70 — Keypad register and interrupt	36
Day 71 — Frontend binding and impossible-input filtering	36
Component 13 — Backup Memory and Saves	37
Day 72 — Save-type detection	37
Day 73 — Static RAM	37
Day 74 — Flash	37
Day 75 — EEPROM	37
Day 76 — Persistence and buffer	38
Component 14 — Audio	39
Day 77 — Architecture and the two subsystems	39
Days 78–80 — The legacy tone channels	39
Days 81–82 — Direct sound and the queue path	39
Day 83 — Mixing and output stage	39
Days 84–86 — Host synchronisation	40
Day 87 — Fast-forward and mute behaviour	40
Day 88 — Buffer and audio validation	40
Component 15 — The BIOS	41
Day 89 — Real BIOS support and boot	41
Day 90 — BIOS skipping and the open replacement	41
Day 91 — Buffer and the system-call inventory	41
Component 16 — Frontend, Presentation, and the SP Screen	42
Day 92 — Window, scaling and presentation	42
Days 93–94 — The SP screen question	42

Day 95 — Quality-of-life features	42
Day 96 — Command-line interface and headless modes	42
Day 97 — Buffer and usability test	43
Component 17 — Accuracy Hardening and Performance	44
Days 98–99 — The deferred accuracy list	44
Day 100 — Profile before optimising	44
Days 101–102 — The optimisations that pay	44
Day 103 — Buffer and compatibility sweep	45
Component 18 — Packaging and Release	46
Day 104 — README as the front door	46
Day 105 — Architecture writeup	46
Day 106 — Demo and screenshots	46
Day 107 — Release engineering	46
Day 108 — Final buffer: done is done	46
Reference: Risk Register	47
Reference: Skills Map	48
Reference: Component-by-Component Timeline Summary	49
Reference: Symptom-to-Cause Triage Table	51

Component 0 — Environment, Repo Foundation & Architectural Commitments

Goal: a working workspace with CI running against a trivial test, and — more important than either — the two architectural decisions that are genuinely expensive to reverse later: your ownership model and your accuracy tier. **Duration:** 3 days.

Day 1 — Toolchain and workspace skeleton

- Confirm the Rust stable toolchain, clippy, rustfmt, and a profiler. Install the profiler now even though you will not use it until Component 17; a missing tool at Day 100 turns into a half-day detour.
- Create the repository with a workspace of at least four crates: the emulator core (no I/O, no windowing, no audio device), a windowed frontend, a headless test runner, and optionally a debugger crate. The core's purity is not an aesthetic preference — it is precisely what makes the automated ROM testing in Component 4 possible, and a core you cannot drive headlessly is a core you cannot write real tests against.
- Configure the release profile aggressively from the start: full link-time optimisation, a single codegen unit, abort on panic. Then raise the *development* profile's optimisation level as well. An unoptimised emulator runs at roughly two percent of target speed, which makes debug builds useless for anything except setting breakpoints. This one line of configuration will save you hours every week for the rest of the project.

Deliverable: *the workspace builds, clippy is clean, and both build profiles are configured.*

Day 2 — The ownership decision

- This is the most consequential Rust-specific decision in the project and it is genuinely hard to reverse once the CPU is written. The problem: the processor calls the bus, the bus needs the scheduler, DMA drives the bus, and the picture unit raises interrupts. In C++ every subsystem simply holds a raw pointer to every other one. Rust will not permit that, so you must choose a model deliberately.
- **Option A — one owning struct with split borrows.** Every subsystem is a field of a single machine struct. The processor holds registers and pipeline state only, never a reference to the bus, and stepping is a method on the machine rather than on the processor. This is the lowest-friction option and it is where most working Rust emulators converge. The cost is one large struct with implementation blocks spread across many modules, which is untidy but harmless.
- **Option B — a generic bus trait.** The processor is generic over a bus interface. This monomorphises away at runtime, so it costs nothing in speed, and it buys you something concrete: the ability to unit-test the processor against a mock bus with no picture unit, no DMA and no timers attached. During Components 2 and 3 that is worth a great deal.
- **Option C — interior mutability throughout.** It works, and you will hit a runtime borrow panic during a DMA transfer that re-enters the bus, in month three, with no obvious cause. Avoid it in the hot path.
- The recommended hybrid is B for the processor specifically, and A for everything else — testability where it matters most, minimal friction everywhere else.
- Write this up as ADR-0001 with the reasoning. It is the single best interview artefact this project produces, because it is a real decision with a real tradeoff that you can defend, rather than a convention you inherited.

Deliverable: *ADR-0001 committed; the ownership model expressed as type definitions that compile.*

Day 3 — Accuracy tier, CI, and the BIOS strategy

- Commit to an accuracy tier and write it into the README on Day 3, not Day 100. **Tier 1, instruction granularity:** cycles accounted after each instruction completes; runs the overwhelming majority of commercial games. **Tier 2, access granularity:** cycles consumed inside each memory access in the order they occur, so timers and DMA observe realistic timing; this is the recommended target and the rest of the plan assumes it. **Tier 3, cycle-accurate bus:** prefetch buffer, sequential access detection, DMA contention — a long tail with real diminishing returns.
- Set up CI running formatting, lint and tests on every push. Create the test-ROM fixtures directory now with a README explaining which ROMs belong there and where to obtain them. Do not commit anything you cannot legally distribute; the directory should be empty in the repository and populated locally and in CI.
- Decide and document the BIOS strategy: require a user-supplied dump, bundle an open-source replacement, or support both. Both is the right answer — the dump is more accurate, the replacement means the emulator works out of the box — but write down why.

Deliverable: *CI green; the README states the accuracy tier and BIOS strategy; the fixtures directory is documented.*

Component complete when: CI is green on an empty project and both architectural commitments are written down rather than held in your head.

Component 1 — Memory Bus & Cartridge

Goal: an address decoder that maps any 32-bit address to the right storage with correct mirroring, reports the cycle cost of every access, and loads a cartridge with its header parsed and validated. Every other component in this plan reads and writes through this one. **Duration: 5 days.**

Day 4 — The address map and region decode

- Implement the decode as a match on the top byte of the address. That single choice gives you a jump table for free and is the shape every fast emulator uses; anything based on range comparisons will show up in your Component 17 profile.
- Ten regions, each with its own size, bus width and access cost. Build the map below into the decoder before you write any of it.

Region	Size	Bus	Notes
BIOS	16 KB	32-bit	Read-only, and read-protected — see Day 6
External work RAM	256 KB	16-bit	Mirrors on a 256 KB boundary; slow, two waits by default
Internal work RAM	32 KB	32-bit	Mirrors on a 32 KB boundary; fast, no waits
I/O registers	~1 KB	32-bit	Not simply mirrored; individual registers have their own rules
Palette RAM	1 KB	16-bit	256 background plus 256 sprite colours
Video RAM	96 KB	16-bit	Irregular mirroring — see Day 5
Object attribute memory	1 KB	32-bit	128 sprite entries with interleaved transform data
Cartridge ROM, waitstate 0	up to 32 MB	16-bit	The same physical ROM as the two regions below
Cartridge ROM, waitstate 1	up to 32 MB	16-bit	Same data, independently configurable timing
Cartridge ROM, waitstate 2	up to 32 MB	16-bit	Same data; serial EEPROM also responds near the top of this region
Cartridge backup	up to 64 KB	8-bit	Static RAM or flash, depending on the cartridge

Bus width matters more than it first appears. External work RAM sits behind a 16-bit bus, so a 32-bit access there is physically two accesses and costs roughly double. Internal work RAM is 32-bit and fast. This distinction is the whole reason games place their hot loops in internal RAM, and it is why your timing will be wrong if you treat all memory as equivalent.

Deliverable: a decoder handling 8-, 16- and 32-bit reads and writes across every region, with unit tests.

Day 5 — Mirroring, and the one case that is not simple

- Most regions mirror on a clean power-of-two boundary: external work RAM every 256 KB, internal every 32 KB, palette and object memory every 1 KB.
- Video RAM does not. It is 96 KB occupying a 128 KB slot, laid out as 64 KB, then 32 KB, then *the same 32 KB again*, with the whole 128 KB pattern repeating above that. Games rely on this. Getting it wrong produces corrupted tiles in specific titles, in a way that looks convincingly like a rendering bug and is not.
- Write a test proving each mirror in both directions: a write at the base is visible at the mirror, and a write at the mirror is visible at the base.

Deliverable: mirroring correct for every region, with the video RAM case explicitly tested.

Day 6 — BIOS read protection and open bus

- The BIOS is readable only while the program counter is currently inside it. Any other read returns the last opcode that was legally fetched from BIOS — a specific stale value, not zero. Several games probe this deliberately, and so do several anti-piracy checks.
- Reads from unmapped addresses return open bus: whatever was last driven on the bus, which in practice is the most recently prefetched instruction. In ARM mode that is straightforward. In THUMB mode the value depends on which region the program counter is in, and for internal work RAM it additionally depends on the alignment of the program counter.
- Stub open bus as returning zero today and record it as accuracy debt. Full fidelity here is not worth pursuing before the processor exists, but it must go on the Component 17 list rather than being quietly forgotten.

Deliverable: *BIOS protection implemented and tested; open-bus behaviour stubbed with a written note of what remains.*

Day 7 — Cartridge loading and header parsing

- Parse the 192-byte header: the entry-point branch, the Nintendo logo bitmap, a 12-byte title, a four-byte game code, a maker code, a fixed verification byte, and a header checksum. Verifying the checksum is a cheap, immediate signal that your loader is reading the file correctly.
- Handle ROM sizing: always a power of two, up to 32 MB, and mirrored across three waitstate regions that address the same physical chip with independently configurable timing.
- Reads past the end of a smaller ROM return a predictable pattern derived from the address rather than zero. A handful of games depend on this and it takes minutes to implement, so do it now rather than debugging it in month four.

Deliverable: *a ROM loads, its title and verified checksum print, and out-of-range reads behave correctly.*

Day 8 — Access timing table and buffer

- Build the waitstate cost table, rebuilt whenever the waitstate control register is written. Waitstate settings select from a fixed set of extra cycles for non-sequential accesses, and sequential access costs differ per ROM region.
- Base costs to encode: internal RAM, I/O and object memory take one cycle at any width; external work RAM takes three cycles narrow and six wide; palette and video RAM take one narrow and two wide; cartridge ROM follows the waitstate table.
- Wire the cost into the access path itself rather than accumulating it separately afterward. This is what makes the Tier 2 accuracy you committed to on Day 3 nearly free later — the timing falls out of the access order naturally instead of having to be reconstructed.

Deliverable: *every access returns data and charges a cycle cost; the cost table rebuilds correctly when the waitstate register is written.*

Component complete when: every region reads, writes and mirrors correctly, BIOS protection works, a real cartridge loads and validates, and every access carries a cycle cost.

Component 2 — CPU Part One: Core Model and the ARM Instruction Set

Goal: the register file with mode banking, the pipeline model, the barrel shifter, and every ARM-mode instruction this machine can execute. This is the longest single stretch in the plan and it produces nothing visible. Expect that. It does not mean the project is going badly. **Duration: 12 days.**

Day 9 — Register file and mode banking

- Sixteen visible registers plus a status register, with per-mode banked copies of some of them and a saved status register per privileged mode.

Processor mode	Entered by	Banked registers
User	Normal program execution	None — this is the base set
System	Privileged code that wants User's registers	Shares User's bank exactly
Supervisor	Reset and software interrupt	Stack pointer, link register, saved status
Interrupt request	A hardware interrupt	Stack pointer, link register, saved status
Fast interrupt	A fast interrupt (unused on this machine)	Five general registers plus stack pointer, link register, saved status
Abort	Memory abort (never raised here)	Stack pointer, link register, saved status
Undefined	An undefined instruction	Stack pointer, link register, saved status

Two traps here. First, User and System share *the same* bank — implementing them as separate banks is a classic bug that only surfaces when a game switches to System mode, which happens late and is confusing when it does. Second, fast interrupt mode banks five additional general registers, so switching into or out of it has to swap those too, even though this machine never actually raises a fast interrupt.

Deliverable: a register file with mode switching, unit-tested by switching modes and confirming banked values survive round trips.

Day 10 — The pipeline model

- Three stages: fetch, decode, execute. The user-visible consequence is that reading the program counter yields the current instruction's address plus eight in ARM mode, and plus four in THUMB mode. Writing it flushes the pipeline and forces two refetches.
- Model this as a two-entry queue that you advance before executing the instruction you pulled from it. Built this way, the program counter already reads correctly during execution and instructions can simply use it, which eliminates an entire category of off-by-eight bugs before it exists.

Deliverable: pipeline advance and flush implemented, with a test confirming the program counter reads correctly in both instruction sets.

Day 11 — Decode strategy

- The ARM instruction class is determined by twelve scattered bits, giving a 4096-entry dispatch table. THUMB needs eight to ten bits, giving 256 or 1024 entries.

- Decide how you generate these. A compile-time constant table has zero startup cost and lands in read-only data; building it lazily on first use is simpler to write. The constant table is the better answer in Rust and is worth the slightly awkward classification logic.
- Note but defer one optimisation: turning decode flags into compile-time constants via const generics lets the optimiser delete branches inside every handler, and is worth a meaningful fraction of core speed. It is also a great deal of boilerplate. Write plain handlers now and revisit in Component 17 only if profiling justifies it.

Deliverable: *both dispatch tables generated, with tests confirming a set of known opcodes route to the correct handler.*

Day 12 — Condition codes and the barrel shifter

- All sixteen condition codes, including the reserved one, which on this architecture generation should be treated as never executing. A failing condition still costs a cycle and still advances the pipeline.
- The barrel shifter is where the bugs live, because its encoding overloads zero and the overload means something different for each operation. A logical or arithmetic right shift encoded as zero actually means a shift of thirty-two. A rotate encoded as zero means rotate-with-extend through the carry flag — an entirely different operation.
- When the shift amount comes from a register instead of the instruction, the rules change again: only the low eight bits are used, an amount of zero now genuinely means no shift at all, amounts at and beyond thirty-two have specific per-operation results for both value and carry, and the instruction takes an extra internal cycle during which the program counter reads twelve ahead rather than eight.
- Write the shifter as a single function with exhaustive tests *before* writing any instruction that uses it. Every data-processing instruction depends on it, so one bug here presents as fifteen unrelated instruction bugs and costs a day to localise.

Deliverable: *a fully tested barrel shifter covering every zero-encoding and every register-amount edge case.*

Days 13–14 — Data processing and status register transfer

- Sixteen data-processing operations covering the logical, arithmetic, comparison and move families.
- Carry and overflow rules differ between addition and subtraction, and the carry flag on subtraction is the *inverse* of borrow. A sign error here propagates into every comparison the game makes, which means it presents as the game behaving strangely rather than as an arithmetic fault.
- Subtract-with-carry and its reversed form compute a value minus another minus the complement of carry. Performing this in wider arithmetic and extracting the flags afterward is far more reliable than trying to get it right in place.
- The status register transfers carry a privilege rule: in User mode, only the flag byte may be written.
- The single most important special case in the entire instruction set: a data-processing instruction with the flag-set bit and the program counter as its destination copies the saved status register back into the current one. This is how every interrupt handler in every game returns. Omit it and interrupts appear to work perfectly until the first one tries to return, at which point the machine wanders into unmapped memory.

Deliverable: *all sixteen operations passing flag tests, with the program-counter-destination status restore tested explicitly and separately.*

Days 15–16 — Load and store, single and halfword

- The alignment behaviour is genuinely strange and none of it is optional. An unaligned word load reads from the aligned address and then *rotates* the result by the misalignment — it does not read across the boundary. An unaligned halfword load rotates by eight bits. An unaligned signed halfword load behaves like a signed byte load on the high byte, which nobody would guess and which real code does hit.
- Stores are simpler: the address is forced to alignment and there is no rotation.
- A load into the program counter flushes the pipeline. On this architecture generation the low bit of the loaded value is ignored and does *not* switch to THUMB — that is a later-generation feature, and implementing it here breaks games.

Deliverable: *every load and store form implemented, with an alignment test suite covering each rotation case.*

Days 17–18 — Block transfer, multiply, swap

- Block transfer is the hardest instruction in the set and it is worth two days on its own. The rules that catch everyone: registers always move lowest-first from the lowest address regardless of the direction bits, so compute the base once and walk upward rather than trying to follow the encoding literally.
- An empty register list transfers the program counter instead, and adjusts the base by the width of all sixteen registers rather than by one.
- Writeback interacts with having the base register itself in the list, and the behaviour differs between load and store, and further depends on whether the base is the first register in the list.
- The user-bank variant transfers to and from User-mode registers regardless of current mode; and with the program counter in the list on a load, it additionally restores the saved status register.
- Multiply timing varies with operand magnitude — one to four internal cycles depending on how many leading bytes of the operand are uniformly zero or one. The carry flag after a multiply is architecturally meaningless on this core; leave it alone rather than inventing a value for it.
- The swap instruction is a read followed by a write with the bus locked, and its data register may legitimately be the same register as its address.

Deliverable: *block transfer passing every documented edge case, multiply with variable timing, and swap implemented.*

Day 19 — Software interrupt and undefined instructions

- Both trap through the exception mechanism you will build properly in Component 3. Wire the dispatch now even if the handler is a stub, so that the plumbing exists when the mechanism arrives.
- Coprocessor instructions have no hardware behind them on this machine and must raise the undefined-instruction exception rather than being silently ignored. Silent ignoring turns a loud, findable fault into a subtle wrong-answer bug.

Deliverable: *both traps dispatch correctly to stub handlers.*

Day 20 — Buffer and first partial test run

- Run the ARM half of a public test suite. You will not pass yet, and that is expected. What you want from today is the *number* of the first failing subtest, which converts a vague sense that something is wrong into a specific, addressable target.
- Catch up on slippage from Days 9 to 19. This component runs long more often than any other in the plan, and the buffer is there for exactly that reason.

Deliverable: *the ARM test ROM runs and reports a specific first failure, with a written list of what remains unimplemented.*

Component complete when: every ARM instruction is implemented, the barrel shifter has exhaustive tests, and the ARM test ROM's failure point has moved deep into the suite.

Component 3 — CPU Part Two: THUMB, Exceptions, and Getting to Green

Goal: the THUMB instruction set, the exception mechanism, and both public processor test suites passing completely. This is the hard gate of the project. **Duration:** 8 days.

Days 21–23 — The THUMB instruction formats

Nineteen formats, all sixteen bits wide, most touching only the low eight registers. Games spend the majority of their execution here because the cartridge's 16-bit bus makes THUMB code roughly twice as fast to fetch as ARM code.

Format	Family	Format	Family
1	Move shifted register	11	Stack-relative load and store
2	Add and subtract	12	Load address from program counter or stack pointer
3	Move, compare, add, subtract immediate	13	Add offset to stack pointer
4	Register-to-register operations	14	Push and pop
5	High register operations and branch-exchange	15	Multiple load and store
6	Program-counter-relative load	16	Conditional branch
7	Load and store, register offset	17	Software interrupt
8	Load and store sign-extended	18	Unconditional branch
9	Load and store, immediate offset	19	Long branch with link (two instructions)
10	Halfword load and store		

- The rules that differ from ARM: nearly all THUMB operations set flags, with narrow exceptions in the high-register and address-computation formats. The program-counter-relative load force-aligns the program counter before adding its offset.
- The long branch with link is genuinely *two separate instructions*, not one wide one. Emulate them independently, because an interrupt can and does fire between the two halves, and a game that takes an interrupt mid-branch will behave incorrectly if you have fused them.
- The empty-register-list quirk from ARM applies here too, in both the push-and-pop and multiple-transfer formats.

Deliverable: all nineteen formats implemented, with the long branch pair handled as two independent instructions.

Day 24 — The exception mechanism

Exception	Mode entered	Return address stored	Raised on this machine?
Reset	Supervisor	Not applicable	At power-on only
Undefined instruction	Undefined	Next instruction	Rarely, but must work
Software interrupt	Supervisor	Next instruction	Constantly — every BIOS call

Exception	Mode entered	Return address stored	Raised on this machine?
Prefetch abort	Abort	Faulting address plus four	Never in practice
Data abort	Abort	Faulting address plus eight	Never in practice
Interrupt request	Interrupt request	Next instruction plus four	Constantly
Fast interrupt	Fast interrupt	Next instruction plus four	Never on this hardware

- The stored return address is the detail that matters most. An interrupt handler returns by subtracting four from the link register; if your stored value is off by four, the game jumps into garbage — but frequently not immediately, which makes this one of the worst bugs in the project to diagnose from symptoms alone.
- Implement all seven vectors even though only two are ever raised here. The unused ones cost almost nothing and their absence would be a silent trap later.

Deliverable: *exceptions entering the correct mode, saving the correct status and return address, and returning correctly.*

Days 25–27 — Getting the test suites to green

- Work the public suites in order: the ARM suite, then the THUMB suite, then the exhaustive per-instruction checker, then randomised fuzzing against a reference implementation.
- These suites halt with a register holding the number of the failing subtest, which makes them ideal for automation: run headless, read the register, assert zero. Wire this into CI now rather than checking manually.
- Three days is a realistic estimate for the fix cycle and it is not padding. The failures you find here are almost entirely in the three categories flagged during Component 2: barrel shifter edge cases, block transfer writeback interactions, and the flag-set-with-program-counter-destination status restore.

Deliverable: *the ARM and THUMB suites pass fully, the exhaustive checker shows all green, and both run in CI.*

Day 28 — Buffer, and a hard gate

This is the gate. Do not pass it with failing processor tests.

Every hour spent here saves roughly ten later. The reason is specific rather than motivational: a graphics bug sitting on top of a processor bug is genuinely ambiguous. When a sprite renders in the wrong place, you cannot tell whether your renderer computed the wrong position or whether the position was already wrong when the game wrote it to memory. Untangling that costs days and it is entirely avoidable by not proceeding until the processor is provably correct.

If you are behind schedule, this is the last component in the plan where the correct response is to extend rather than to cut.

Deliverable: a green processor.

Component complete when: both instruction-set suites pass in CI and you would bet money that the processor is correct.

Component 4 — Verification Harness and Debug Tooling

Goal: the tooling that turns every later bug from a multi-day mystery into a twenty-minute diagnosis. This is the component people skip in order to get to graphics faster, and then regret for the remaining four months.

Duration: 5 days.

Day 29 — Headless runner and the ROM test harness

- A headless binary that loads a ROM, runs a fixed number of frames, and exits with a status derived from the machine's final state. Everything else in your testing story builds on this, and it only exists because Component 0 kept the core free of I/O dependencies.
- Turn each public test ROM into a test case behind a macro, so that adding a new suite is a single line rather than a new harness.

Deliverable: *the full public suite runnable as a normal test command, executing headlessly in CI.*

Day 30 — Trace diffing

- This is the single most valuable technique in the project and it is worth building carefully.
- Emit one line per executed instruction containing all sixteen registers and the status register, formatted to match your reference emulator's trace output exactly. Run both against the same ROM, diff the two files, and find the first divergence. The instruction immediately before the divergence is almost always the bug.
- Traces become enormous very quickly, so add two controls now rather than later: begin tracing only after a specified number of instructions, and maintain a ring buffer of the last several thousand instructions that is dumped automatically on a panic.

Deliverable: *a trace format that diffs cleanly against your reference emulator, demonstrated by deliberately introducing a bug and locating it.*

Day 31 — Disassembler

- Not optional. You will read a great deal of ARM assembly over the next three months and reading raw hexadecimal is not a viable way to do it.
- Cross-check its output against the standard cross-toolchain's disassembler on a real ROM. Any disagreement is either a disassembler bug or a decode bug, and both are worth finding today rather than in month three.

Deliverable: *a disassembler for both instruction sets, validated against the reference toolchain.*

Day 32 — Save states

- Add these before the picture unit, not after. They cut iteration time dramatically: being able to jump straight to the frame where a game misbehaves, rather than replaying five minutes of intro each time, is worth several days across the remainder of the project.
- The fiddly part is the scheduler. Store event times relative to the current cycle count so that a loaded state remains internally consistent.
- Write the determinism test today: save a state, run a thousand frames, hash the framebuffer; reload the state, run a thousand frames, hash again. Any mismatch means some piece of machine state is not being captured, and finding that now is far cheaper than finding it in month four when the state is ten times larger.

Deliverable: *save states that round-trip, with a passing determinism test in CI.*

Day 33 — Debugger skeleton and buffer

- A minimal interactive debugger: register view across all modes, disassembly around the program counter, breakpoints, a memory viewer, and single-stepping. The graphics-specific views — layer toggles, tile viewer, palette viewer, sprite attribute viewer — are added during Components 9 through 11, when there is finally something to view.
- Consider implementing the standard remote debugging protocol instead. It is roughly three hundred lines and it gives you a full-featured debugger for free, including one you can also point at your reference emulator for side-by-side inspection.

Deliverable: *a debugger you can actually step instructions in.*

Component complete when: a deliberately introduced processor bug can be located by trace diff in under half an hour.

Component 5 — Timing and the Scheduler

Goal: replace per-cycle ticking of every subsystem with an event queue. This is simultaneously the largest performance decision and the largest accuracy decision in the project. **Duration: 4 days.**

Day 34 — The timing model

Quantity	Value	Derivation
System clock	16,777,216 Hz	Exactly two to the twenty-fourth — useful, because many divisions come out clean
Cycles per dot	4	Fixed
Dots per scanline	308	240 visible plus 68 horizontal blanking
Cycles per scanline	1,232	308 dots at four cycles each
Scanlines per frame	228	160 visible plus 68 vertical blanking
Cycles per frame	280,896	1,232 multiplied by 228
Refresh rate	about 59.73 Hz	Clock divided by cycles per frame — not 60

- Note what the naive design costs before you reject it. Ticking four subsystems every cycle means roughly sixty-seven million function calls per emulated second, almost all of which do nothing. The event-driven alternative computes when the next interesting thing is due and lets the processor run freely until then.

Deliverable: the timing constants defined, with a test asserting the frame cycle count is exactly right.

Day 35 — Scheduler implementation

- A priority queue of timed events against a monotonic global cycle counter.
- Ties must break deterministically, by insertion order. If they do not, your save-state determinism test from Component 4 will begin failing intermittently for reasons that take a full day to track down, because the failure will look like a state serialisation bug rather than an ordering one.
- Note for Component 17: a binary heap is the obvious structure, but this machine rarely has more than a dozen pending events, and a short sorted vector usually outperforms it. Measure then rather than guessing now.

Deliverable: a scheduler with push, peek and dispatch, plus a determinism test on tie-breaking order.

Day 36 — The main loop

- Order matters and it is not arbitrary: dispatch everything due at the current time, then run DMA if any channel is active (DMA halts the processor), then check for pending interrupts, then step the processor.
- Cycles are consumed inside memory accesses rather than added afterward, which is precisely what makes the Tier 2 accuracy you committed to fall out of the design naturally.
- Implement halt fast-forwarding here. A halted processor should jump the clock straight to the next scheduled event rather than stepping cycle by cycle. Games spend a great deal of their time halted waiting for vertical blank, so this is one of the largest and cheapest speed wins available anywhere in the project.

Deliverable: a main loop running exactly one frame's worth of cycles, with halt fast-forwarding working.

Day 37 — Buffer and accuracy-tier check

- Re-read the accuracy tier you committed to in Component 0 and confirm the scheduler you have actually built supports it. If you chose Tier 2 but cycles are being accumulated after instructions complete rather than during individual accesses, fix it today. Retrofitting this later means touching every instruction handler you wrote over the last month.

Deliverable: *the scheduler matches the committed accuracy tier, verified by a frame-count test.*

Component complete when: exactly 280,896 cycles elapse per frame, events fire in deterministic order, and a halted processor costs almost nothing.

Component 6 — Interrupts and Timers

Goal: the interrupt controller and four timers, both driven by scheduled events rather than by polling. **Duration:** 4 days.

Day 38 — The interrupt controller

- Three registers: an enable mask, a set of request flags, and a master enable. Fourteen sources, listed below.

Source	Raised by	Source	Raised by
Vertical blank	Picture unit, once per frame	Timers 0 to 3	Timer overflow, one source each
Horizontal blank	Picture unit, per visible line	Serial	Link cable transfer complete
Line match	Picture unit, at a configured line	DMA 0 to 3	Transfer complete, one source each
Keypad	A configured key combination	Cartridge	Cartridge removal

- The rule that breaks the most first attempts:** request flags are acknowledged by writing a *one* to the bit you want to clear. Writing zero does nothing at all. Getting this backwards means handlers never clear their flag, fire immediately again on return, and the game freezes in a way that looks convincingly like a processor bug.
- The second rule:** halt wakes on an enabled pending interrupt *regardless of the master enable*. A game can legitimately halt with interrupts masked, wake on vertical blank, and continue without ever entering a handler. Requiring the master enable in your wake condition hangs those games permanently and there is no obvious symptom pointing at the cause.

Deliverable: interrupts raising, dispatching and acknowledging correctly, with halt-wake behaviour tested explicitly.

Day 39 — Timers, scheduled

- Four timers sharing a set of prescaler divisions, with an optional cascade mode in which a timer counts overflows of the previous timer rather than clock ticks.
- The register trap:** reading the count register and writing it touch two *different* physical registers. Reads return the live counter; writes set the reload value used on the next overflow. Treating them as one register produces bugs that appear only in games that read their timers back, which is a subset large enough to matter and small enough to be confusing.
- Compute overflow times and schedule them rather than ticking each timer every cycle. Reading the counter then means deriving its value from elapsed time rather than storing it.

Deliverable: four timers with prescalers, cascade mode and overflow interrupts, all event-scheduled.

Day 40 — Timer edge cases

- Enabling a timer reloads its counter, with a short startup delay before counting actually begins.
- The prescaler is a free-running divider off the system clock and is *not* reset when a timer is enabled. A timer with a large prescaler therefore does not necessarily overflow a clean multiple of cycles after being enabled. Aligning to the global clock rather than to the enable time is the accurate model, and audio code in Component 14 depends on it.
- Cascade mode ignores the prescaler entirely.

- Writing the reload value while a timer is running does not change the current count, only what it reloads to on its next overflow.

Deliverable: *the public timer test ROM passing, or a homebrew cycle-counter reporting the correct cycles per frame.*

Day 41 — Buffer

- Catch up on slippage and add both subsystems' tests to CI.

Deliverable: *interrupts and timers closed out with tests running in CI.*

Component complete when: a test ROM that enables a vertical-blank interrupt, halts, and counts handler invocations shows exactly one per frame.

Component 7 — DMA

Goal: four DMA channels with correct priority, preemption, timing triggers, and the special modes that the audio and video hardware depend on. **Duration:** 4 days.

Day 42 — Channel model and per-channel differences

Four channels, each with a source, a destination, a transfer count and a control register. They are emphatically not interchangeable:

Channel	Priority	Source	Destination	Special timing means
0	Highest	Internal memory only	Internal memory only	Prohibited
1		Any	Internal memory only	Audio queue refill
2		Any	Internal memory only	Audio queue refill
3	Lowest	Any	Any, including cartridge	Video capture

- Control settings cover source and destination stepping (increment, decrement, fixed, or increment-with-reload), repeat, transfer width, start timing, and an interrupt on completion.
- A transfer count of zero means the channel's maximum, not zero transfers — and the maximum differs between channel three and the others.

Deliverable: all four channels configurable, with their per-channel restrictions enforced.

Day 43 — Transfer execution and priority

- Lower-numbered channels have higher priority and *preempt* a running lower-priority transfer mid-flight. Your transfer routine must therefore be interruptible rather than a tight loop that runs to completion — this constrains the shape of the code, so get it right the first time.
- The first access of a transfer is non-sequential and the remainder are sequential, which matters for the cycle costs you built in Component 1.
- DMA halts the processor while it runs. Your main loop already accounts for this if Component 5 was built in the specified order.

Deliverable: transfers executing with correct priority and preemption, verified by a test that starts two channels simultaneously.

Day 44 — Timing triggers and the special modes

- Four start timings: immediate, vertical blank, horizontal blank, and a per-channel special mode.
- Immediate DMA does *not* start instantly. There is a short startup delay, and games that write the control register and then immediately read the destination depend on it. Schedule the transfer rather than running it inline inside the register write.
- Repeat is ignored for immediate transfers — the enable bit always clears afterward.
- The special mode means audio queue refill on channels one and two, and video capture on channel three. Queue refills always move four 32-bit words to a fixed destination regardless of what the count and width settings say. This is hardwired behaviour, not a configurable case, and implementing it as configurable will produce audio that is subtly wrong in a way you will not diagnose until Component 14.
- Source addresses in the BIOS region are ignored; DMA cannot read the BIOS.

Deliverable: *all four timings working, with the immediate-mode startup delay and the audio queue special case implemented.*

Day 45 — Buffer and trigger validation

- Confirm horizontal-blank DMA fires exactly 160 times per frame and vertical-blank DMA exactly once. Both are easy to get subtly wrong — firing during blanking lines, or being one line out — and both produce symptoms far from the cause.

Deliverable: *the DMA test ROM passing and per-frame trigger counts verified.*

Component complete when: DMA passes its public tests and the audio queue refill path is in place, ready for Component 14.

Component 8 — PPU Part One: Display Timing and Bitmap Modes

Goal: the video state machine, the framebuffer, and the three bitmap modes — the fastest possible route from fifty days of test output to something you can actually look at. **Duration:** 5 days.

Day 46 — The rendering architecture decision

Three options, and this choice constrains everything that follows it. Make it deliberately and write it down.

Approach	Accuracy	Cost	Verdict
Per-frame	Breaks every raster effect	Lowest	Do not choose this. Games change display registers mid-frame constantly, and a per-frame renderer cannot see those changes at all.
Per-scanline	Handles mid-frame register changes correctly	Moderate	The right choice. Render each complete line at horizontal blank using the register values at that instant. This is what most emulators do.
Per-pixel	Highest	Substantiall y higher	Needed for a small number of demos that change registers mid-line. Not worth it at this stage; note it as a possible future direction.

- Choose scanline rendering and record the decision along with what it cannot do — it cannot handle a register write that lands part-way through a scanline, which a very small number of programs rely on. Naming the limitation yourself is worth more than hoping nobody asks.

Deliverable: an ADR recording the scanline choice and its known limitation.

Day 47 — The video state machine

- Two events per line: horizontal blank begins, and the line ends. Render the visible line when horizontal blank begins; advance the line counter and evaluate the line-match comparison at line end.
- Two timing details worth knowing early even if you start with the naive values. First, the horizontal blank flag is set roughly 1,006 cycles into the 1,232-cycle line, not immediately after the last visible dot at 960. Second, the vertical blank flag clears one line *early* — on the final line of the frame rather than at the wrap. Starting with the obvious values works for most games; record both as refinements for Component 17.
- Horizontal-blank DMA fires on visible lines only, not during vertical blanking.

Deliverable: a state machine producing 228 line transitions and one vertical blank per frame, with line-match interrupts firing at the configured line.

Day 48 — Colour and the framebuffer

- Colours are fifteen bits, blue-green-red ordered, five bits per channel. Converting to eight bits per channel by shifting left three is subtly wrong: it never reaches full white, so every bright scene in every game is slightly dim. Replicate the high bits into the low ones instead.
- Palette memory holds 256 background colours and 256 sprite colours. In four-bit mode these are sixteen palettes of sixteen colours each, and index zero of any palette is transparent — which means transparency cannot be represented by a colour value alone.

- Carry opacity as information separate from colour from the very start. Retrofitting it during the blending work in Component 11 is genuinely painful, because by then a dozen render paths have baked in the assumption.

Deliverable: *colour conversion with a test proving full-scale white, and a framebuffer the frontend can read.*

Day 49 — The three bitmap modes

Mode	Resolution	Colour	Buffering
3	Full, 240 by 160	Direct 16-bit	Single buffer — no page flipping
4	Full, 240 by 160	8-bit palette indices	Two switchable frames
5	Reduced, 160 by 128	Direct 16-bit	Two switchable frames

- All three render through background layer two, and all three pass through that layer's affine transform — which means bitmap modes can be rotated and scaled. Implement the transform path now, even though affine backgrounds proper arrive in Component 9. Building it twice is worse than building it once slightly early.
- Direct-colour modes have no transparency concept: every pixel is present. This is exactly why opacity has to be tracked separately from colour, and today is where that decision first pays off.

Deliverable: *all three bitmap modes rendering correctly, verified against a homebrew bitmap demo.*

Day 50 — Buffer and first visible output

- Wire up a minimal window that displays the framebuffer. It does not need to be good; it needs to exist. After roughly fifty days of test-suite output, seeing an actual image is worth something real to the project's chances of being finished.

Deliverable: *a bitmap-mode homebrew demo visibly rendering in a window.*

Component complete when: **bitmap demos render correctly and the line timing produces the right interrupt counts per frame.**

Component 9 — PPU Part Two: Tiled Backgrounds

Goal: the four text backgrounds and the affine backgrounds — the layers that essentially every commercial game actually uses. **Duration:** 6 days.

Day 51 — Background control and tile addressing

- Each background has a control register selecting its priority, character data base, tilemap base, colour depth, screen size and mosaic participation.
- Tilemap entries are sixteen bits: a tile index, horizontal flip, vertical flip, and — in four-bit mode only — a palette number.
- Character data and tilemaps both live in background video RAM and are addressed in blocks: character bases in 16 KB units, screen bases in 2 KB units.
- Backgrounds cannot read sprite video RAM. An address that runs past the background region must produce nothing rather than reading whatever happens to be there — and the difference is visible in real games.

Deliverable: control register decoding and address computation, tested against known values.

Days 52–53 — Text background rendering

Screen size	Dimensions	Storage layout
0	256 by 256 pixels	One 32-by-32 tile screen block
1	512 by 256 pixels	Two screen blocks side by side
2	256 by 512 pixels	Two screen blocks stacked
3	512 by 512 pixels	Four screen blocks in a two-by-two arrangement

- The two wide sizes are the trap. A 512-pixel-wide map is *not* one contiguous 64-tile-wide table — it is stored as separate 32-by-32 tile screen blocks laid out in a specific order, so the index computation has to account for which block a given coordinate falls in.
- The symptom when this is wrong is distinctive and worth recognising in advance: a correct-looking left half of the screen and a garbled right half.
- Scroll offsets wrap within the map dimensions, not within the screen dimensions.
- Both colour depths, with the four-bit case requiring correct nibble ordering within each byte — getting the nibbles backwards produces colourful noise that looks like a palette bug.

Deliverable: text backgrounds rendering correctly at all four screen sizes, with an explicit wide-map test.

Days 54–55 — Affine backgrounds

- Backgrounds two and three can be affine in the appropriate display modes, transformed by four fixed-point matrix parameters plus a reference point.
- **The critical behaviour:** the reference point registers you write are not what is used per scanline. Internal accumulator registers are latched from them at the start of vertical blank, and thereafter advance by two of the matrix parameters after every scanline. Writing the reference registers mid-frame updates the internal accumulators immediately.

- That mechanism is precisely how games produce per-scanline warping and pseudo-three-dimensional road effects, so it is not an obscure corner — it is the whole point of affine backgrounds in practice. Model the latch-and-accumulate behaviour rather than recomputing from the registers each line, which will look correct on a static screen and wrong on every effect.
- Affine backgrounds are always eight-bit colour, always use one-byte tilemap entries, and never flip. They also have an optional wraparound mode; with it off, out-of-bounds pixels are transparent rather than clamped.

Deliverable: *affine backgrounds rendering with correct latch-and-accumulate reference behaviour, verified against a rotating homebrew demo.*

Day 56 — Buffer and first title screens

- Add the picture-unit debug views now that there is something to view: per-layer enable toggles, a tile viewer, and a palette viewer.
- The layer toggles are the important one and they are worth more than they sound. Being able to switch each background off individually converts 'the screen looks wrong' into 'background two is wrong', which is the difference between a day of guessing and twenty minutes of looking.

Deliverable: *a commercial game reaching a recognisable title screen, and layer toggles working in the debugger.*

Component complete when: text and affine backgrounds render correctly and at least one real game shows a correct title screen.

Component 10 — PPU Part Three: Sprites

Goal: 128 sprites with the full range of sizes, flipping, affine transforms, priority handling, and correct interaction with the background layers. **Duration:** 6 days.

Day 57 — Object memory and the attribute layout

- 128 entries of eight bytes each — but only the first six bytes of each entry are sprite attributes. The remaining two hold affine matrix parameters, interleaved across groups of four entries. This interleaving is not an implementation detail you can abstract away; the parameters genuinely live in the gaps and must be gathered from their scattered locations.

Attribute	Carries
Attribute 0	Vertical position, rotation enable, double-size or disable, sprite mode (normal, semi-transparent, or window), mosaic, colour depth, shape
Attribute 1	Horizontal position, and then either an affine parameter group index (when rotation is enabled) or horizontal and vertical flip flags (when it is not), plus size
Attribute 2	Tile index, priority, and palette number in four-bit mode

- Shape and size combine into a table of twelve possible pixel dimensions, from eight-by-eight up to sixty-four-by-sixty-four including wide and tall variants.
- One overloaded bit worth flagging now:** attribute zero's ninth bit means 'double the bounding box' when rotation is enabled, and 'disable this sprite entirely' when it is not. Two completely unrelated meanings in one bit, selected by another bit.

Deliverable: attribute parsing including the interleaved affine parameter groups, tested against known object memory contents.

Days 58–59 — Sprite rendering

- Coordinates are awkward in two different ways. Horizontal position is nine-bit signed, so sprites hanging off the left edge of the screen are entirely routine. Vertical position is eight-bit and wraps, so a sprite near the top of the screen may have a stored value near 256. Handle both or sprites will appear to teleport across the display at the screen edges.
- Tile addressing has two modes: one-dimensional, where a sprite's tiles are contiguous in memory, and two-dimensional, where each row of tiles wraps at a fixed stride. The mode is a global display-control bit rather than a per-sprite one, which is easy to miss.
- Priority ties resolve by object memory index, with the lower index winning. Sprites also draw above backgrounds at the same numeric priority. Getting the tie-break wrong produces flickering that looks convincingly like a timing bug and is not.

Deliverable: sprites rendering with both mapping modes, correct coordinate wrapping, and correct tie-breaking.

Day 60 — Affine sprites

- The same four-parameter transform as affine backgrounds, applied around the sprite's centre, with an optional doubled bounding box so that a rotated sprite is not clipped by its own un-rotated bounds.

- The transform maps *screen* position back to *texture* position, not the other way around. Running it forward produces visible gaps between pixels wherever the transform magnifies, which is the classic symptom of having the direction inverted.

Deliverable: *affine sprites rotating and scaling correctly, including the double-size bounding box case.*

Day 61 — Sprite video RAM restrictions and edge cases

- In bitmap display modes the bitmap occupies the first 512 tiles of sprite video RAM, so only higher tile indices are usable and lower ones must render nothing at all. Games that place sprites over a bitmap depend on exactly this behaviour.
- In eight-bit colour with two-dimensional mapping, tile indices are effectively halved.
- Note but defer: sprites are limited by a per-scanline cycle budget, which is reduced when the display-control bit that frees object memory during horizontal blank is set. Most games stay comfortably under it; a few exceed it deliberately as a visual effect. Log this as accuracy debt for Component 17 rather than implementing it now.

Deliverable: *bitmap-mode sprite restrictions implemented, and the cycle budget recorded as deferred debt.*

Day 62 — Buffer and the sprite viewer

- Add an object-attribute viewer to the debugger showing every sprite's decoded attributes alongside a preview of its tile data. This pays for itself the first time a sprite is invisible and you need to know whether it is disabled, off-screen, or pointing at the wrong tiles.

Deliverable: *sprites correct in at least two commercial games, and the sprite viewer working.*

Component complete when: a sprite-heavy game renders its sprites correctly with proper layering against the backgrounds.

Component 11 — PPU Part Four: Windows, Blending, and Mosaic

Goal: the compositing effects layer — the part that turns 'the right things are on screen' into 'this looks like the real machine'. **Duration:** 7 days.

Days 63–64 — Windows

Region	Defined by	Precedence
Window 0	A rectangle, when enabled	Highest
Window 1	A second rectangle, when enabled	Second
Object window	Opaque pixels of sprites in window mode	Third
Outside	Everything not covered above	Lowest — the fallback

- Each region independently controls which of the five layers are visible inside it *and* whether blending is permitted there. That second control is how games confine an effect to a region, and it is easy to overlook when reading the register description.
- The rectangle bounds have quirks worth handling deliberately: right and bottom edges are exclusive, and a right edge greater than the screen width, or numerically less than the left edge, clamps to the screen edge rather than producing an empty or wrapped region.
- The object window is defined by sprites in a specific mode, and those sprites are never drawn — they contribute only their opaque pixels to a mask. This means your sprite renderer needs a separate output path for them, which is much easier to add now than to retrofit.

Deliverable: *all four window regions with correct precedence and correct handling of the edge-case bounds.*

Days 65–66 — Blending

- Three effects: alpha blending between two layers, brightening toward white, and darkening toward black. Layer participation is selected per layer as a first or second blend target.
- **The rule that surprises people:** a sprite in semi-transparent mode alpha-blends with whatever is beneath it *regardless of what the blend control register says the current effect is*. It overrides the configured effect entirely. Getting this wrong makes shadows, ghosts and water render as solid blocks — a very recognisable symptom once you have seen it once.
- Alpha blending also requires the second target to be the layer *immediately* below the first at that specific pixel. If nothing qualifying is directly beneath, no blend occurs and the first target simply draws normally — it does not fall through and blend with the backdrop.
- The backdrop, meaning palette entry zero, is itself a valid blend target.
- Brightening and darkening apply to the first target only and need no second target at all.

Deliverable: *all three effects working, with the semi-transparent sprite override and the immediately-below rule both tested.*

Day 67 — Composition

- Assemble the per-pixel pipeline: determine the window mask, collect the top two visible layers by priority (with sprites taking precedence over backgrounds at equal priority), fall back to the backdrop, then apply effects.
- Skip disabled layers before rendering them rather than rendering and discarding. This is free performance and it simplifies the priority walk, which is worth having simple because it is where the remaining compositing bugs will hide.

Deliverable: *the full compositor producing correct output on games using layered effects.*

Day 68 — Mosaic

- Two independent mosaic sizes, one for backgrounds and one for sprites, applied only to layers that explicitly opt in via their control registers.
- Vertical mosaic on backgrounds snaps the *source* row, so several output rows sample the same source line. It is not a post-process applied to the finished output, and implementing it as one produces results that are close but visibly wrong during transitions.

Deliverable: *mosaic working on both backgrounds and sprites, verified against a game that uses it for screen transitions.*

Day 69 — Buffer and picture-unit validation sweep

- Run the public picture-unit test suites and compare framebuffer hashes against known-good references. Add these to CI as hash assertions — they are the only practical way to detect a rendering regression automatically.
- Work through a spread of visually demanding games: an affine-heavy role-playing game, a racer using per-scanline register writes, and a title using heavy blending. Three games covering three different stress patterns will find more than ten games covering one.

Deliverable: *picture-unit test suites passing, framebuffer hash tests in CI, and three visually demanding games rendering correctly.*

Component complete when: the picture-unit test suites pass and games using windows, blending and affine effects look right.

Component 12 — Input

Goal: the keypad. Two days, the shortest component in the plan, and the one where the project stops being a technical exercise and becomes something you can play. **Duration: 2 days.**

Day 70 — Keypad register and interrupt

- Ten keys in a single register, **active low** — a set bit means the key is *not* pressed. Inverting this is essentially the entire bug surface of the component, and it presents as every input doing the opposite of what you expect, which is at least easy to spot.
- The keypad control register selects which keys participate and whether the interrupt fires on any selected key or only when all of them are held.
- The keypad interrupt works even in the low-power stop state, which is precisely how that state is woken.

Deliverable: *input reaching the emulated machine, with keypad interrupts firing on the correct condition.*

Day 71 — Frontend binding and impossible-input filtering

- Map keyboard and gamepad input with configurable bindings.
- Filter physically impossible directional combinations. A real directional pad cannot press left and right simultaneously; a keyboard can, and a handful of games glitch or crash when they see it because the situation is genuinely unreachable on real hardware. Mask them in the frontend rather than in the core.

Deliverable: *a playable game with configurable bindings and impossible input combinations filtered.*

Component complete when: you can play a commercial game start to finish with a keyboard or gamepad.

Component 13 — Backup Memory and Saves

Goal: all three cartridge save technologies, correctly detected and reliably persisted. **Duration:** 5 days.

Day 72 — Save-type detection

- There is no header field identifying the save type. The standard approach is to scan the ROM for the signature string that the cartridge's save library leaves behind, checking the more specific flash variants before the general one.
- Build a game-code override table alongside the scanner. A meaningful number of games have missing, wrong, or multiple contradictory signatures, and every mature emulator maintains such a list. Accepting that you need one is a scoping decision worth documenting rather than a problem worth fighting.

***Deliverable:** detection working across a sample of real games, with an override table in place and populated.*

Day 73 — Static RAM

- The simplest of the three: a fixed block of battery-backed memory behind an eight-bit bus.
- The bus width has visible consequences. Wider reads return the byte duplicated across all lanes, and wider writes write only the byte matching the address alignment. Treating it as ordinary memory produces saves that appear to work and are subtly corrupt.

***Deliverable:** static-RAM saves working, with the eight-bit bus behaviour correct.*

Day 74 — Flash

- Two sizes, the larger of which is bank-switched. Access is by command sequence written to two magic addresses rather than by direct writes — so a naive memory-mapped implementation will silently do nothing at all.
- The command set covers entering and leaving chip-identification mode, preparing an erase, erasing the whole chip, erasing a single sector, writing a byte, and selecting a bank.
- Games read the chip identification to decide which flash routine to use, so the identifier you report must match the size you claim to be. A mismatch here produces a game that appears to save and then cannot load.
- Flash writes can only clear bits; erasing is what sets them back. Modelling the write as a bitwise operation rather than a plain assignment is more accurate and costs nothing.

***Deliverable:** both flash sizes working with correct chip identifiers, tested against real games of each type.*

Day 75 — EEPROM

- The strangest of the three, and the one that most often takes an extra day. EEPROM is not memory-mapped at all — it is a serial device driven by DMA transfers to the top of the cartridge address space, one bit per transferred halfword.
- Two sizes, distinguished by address width. You usually cannot tell them apart in advance. The standard trick is to infer the size from the length of the first DMA transfer directed at the EEPROM region, since the address width determines how many bits a command occupies.
- For larger ROMs, EEPROM responds only in a narrow window at the very top of the region rather than across the whole of it.

Deliverable: both EEPROM sizes working with automatic size detection, tested against real games.

Day 76 — Persistence and buffer

- Write saves atomically: to a temporary file, then rename. A crash part-way through a write must never corrupt a save file, because people will use this on real playthroughs and a corrupted sixty-hour save is not recoverable.
- Flush on a timer whenever the save is dirty, as well as on clean exit. A crash should cost seconds of progress, not hours.
- Use the conventional file extension and raw binary contents so that saves interoperate with other emulators. Interoperability here is genuinely expected by users and costs you nothing.

Deliverable: saves persisting across restarts for at least one game of each save type, written atomically.

Component complete when: four games spanning all three save technologies save and load reliably across restarts.

Component 14 — Audio

Goal: both audio subsystems, mixed, resampled and synchronised to the host sound device. Audio is where every timing bug elsewhere in the machine stops being invisible and starts being audible. **Duration: 12 days.**

Day 77 — Architecture and the two subsystems

Subsystem	Channels	Nature	Fed by
Legacy tone	4	Two square waves (one with frequency sweep), a wavetable channel, and a shift-register noise channel	Register writes, clocked by an internal frame sequencer
Direct sound	2	Eight-bit signed sampled audio from 32-byte queues	DMA refill, clocked by a timer overflow popping one sample per tick

- The direct sound channels are how every game plays its music and speech. The legacy channels are inherited essentially unchanged from the previous console generation.
- If you have written an emulator for that previous generation, the four legacy channels are largely reusable and this component compresses considerably. If not, budget the extra days honestly rather than hoping.

Deliverable: the audio module structure in place, with both subsystems stubbed and registers decoding.

Days 78–80 — The legacy tone channels

- Each channel has some combination of a length counter, a volume envelope, and — on one channel — a frequency sweep. All are driven by a frame sequencer that clocks through eight steps at a fixed rate, performing different operations on different steps.
- Square waves have four selectable duty patterns. The wavetable channel has two banks that can be used separately or combined into one longer waveform, which is an addition over the previous generation and is easy to miss. Noise uses a shift register with two selectable widths.
- Verify against a reference emulator by ear on the same passage. Waveform bugs are far easier to hear than to see in a trace.

Deliverable: all four tone channels producing correct waveforms, verified against a reference.

Days 81–82 — Direct sound and the queue path

- Each queue is refilled by whichever DMA channel is configured to target it in the special timing mode you implemented in Component 7. The queue holds a limited number of samples and requests a refill when it drains past a threshold.
- Underrun behaviour matters more than it sounds: when the queue empties, the output holds the last sample rather than dropping to silence. Getting this wrong produces clicking whenever the game is under load.
- Each queue is clocked by one of the first two timers, selected per channel. This is the point in the project where every timing bug you have anywhere in the machine becomes audible as wrong pitch or wrong tempo — which is genuinely useful diagnostic information, if unwelcome.

Deliverable: direct sound playing sampled audio from a real game at the correct pitch.

Day 83 — Mixing and output stage

- Mix the four tone channels and the two direct channels according to their independent volume settings and left-right routing, then apply the output bias and clamp.
- Choose a sample rate that divides the system clock exactly. An exact divisor eliminates accumulated rounding drift entirely, which is worth more than picking a conventional rate and resampling from the start. Resample to the host's preferred rate afterward if you need to.

Deliverable: *a mixed stereo stream at a clock-exact sample rate.*

Days 84–86 — Host synchronisation

Budget the full three days. This is consistently underestimated.

The core problem: your emulator produces samples on emulated time and the sound device consumes them on real time. These two rates will drift apart, and no amount of getting the emulation right prevents it.

The wrong fix is to pace the emulator against a wall clock and let the audio buffer run dry when it falls behind. This is what produces the crackling that people associate with amateur emulators, and it cannot be tuned away because the underlying loop is wrong.

The right fix is to pace the emulator against the audio device itself: run frames only as fast as the buffer drains. Use a lock-free ring buffer between the emulator and the audio callback, and on underrun output silence rather than repeating the previous buffer — repeats are considerably more audible than gaps.

Then add dynamic rate control: nudge the resampling ratio slightly based on how full the buffer is, marginally faster when it is over half full and marginally slower when under. The adjustment is inaudible and it eliminates both crackling and long-term drift. This is precisely the difference between audio that is technically correct and audio that sounds right.

Deliverable: *a full minute of game music with no crackling, no drift and no underruns.*

Day 87 — Fast-forward and mute behaviour

- Decide what audio does when the emulator runs uncapped. Muting is simplest; resampling the accelerated stream is better but more work. Either is a defensible choice.
- What is not optional: the synchronisation loop must not deadlock waiting for buffer space that is being consumed at normal speed while frames are produced at ten times normal. Test this deliberately rather than discovering it from a user report.

Deliverable: *fast-forward working without audio deadlock or artefacts.*

Day 88 — Buffer and audio validation

- Test against games using different sound engines — the major first-party engines behave quite differently from each other, and a bug that is inaudible in one is obvious in another.

Deliverable: *audio validated on three games across different sound engines.*

Component complete when: *a game's music plays cleanly for several minutes with no drift, and both subsystems are clearly audible.*

Component 15 — The BIOS

Goal: a working boot path that respects the legal constraints established on Day 3, with a documented strategy users can understand. **Duration:** 3 days.

Day 89 — Real BIOS support and boot

- With a real dump, boot from the reset vector and let the BIOS do its own work: validate the cartridge logo, set up the stacks for each mode, and jump to the cartridge entry point.
- Verify user-supplied dumps by hash and fail with a clear message naming the problem, rather than booting to a black screen and leaving the user to guess.

Deliverable: *games booting from the reset vector with a real BIOS.*

Day 90 — BIOS skipping and the open replacement

- Implement a fast-boot path that sets up the machine state the BIOS would have left behind: the stack pointers for the relevant modes, the initial processor mode, the sound bias register, and the cartridge entry point.
- Integrate a clean-room open-source BIOS replacement as the bundled default so the emulator works out of the box for someone who has no dump. Compatibility is very good but not perfect; say so in the README rather than letting a user discover it.
- **Note the third option and why this plan does not recommend it.** High-level emulation — intercepting the system calls and implementing them natively, along with the interrupt dispatcher — works for most games and fails for anything that reads BIOS memory directly, uses undocumented calls, or depends on BIOS timing. Since a real dump always beats it and a bundled replacement covers the out-of-box case, the effort is better spent in Component 17.

Deliverable: *three boot paths — real BIOS, bundled replacement, and skip — all producing identical behaviour after the first frame.*

Day 91 — Buffer and the system-call inventory

- Even with a real BIOS you should know what the calls are, because they appear constantly in traces and recognising them is the difference between reading a trace and decoding one.
- Build a reference table of the roughly thirty calls: soft reset, memory clearing, halt and stop, interrupt waiting, division, square root, arc tangent, block copy and fill, affine parameter computation, several distinct decompression formats, and the sound driver entry points.
- Note one behaviour that games genuinely hit: division by zero has defined results on this hardware rather than trapping, and a game relying on that will behave incorrectly if you trap.

Deliverable: *a system-call reference table committed to the documentation.*

Component complete when: all three boot paths work and the BIOS strategy is documented for users.

Component 16 — Frontend, Presentation, and the SP Screen

Goal: the parts a user actually touches — and the one component in the entire plan where 'SP' genuinely means something different from 'GBA'. **Duration:** 6 days.

Day 92 — Window, scaling and presentation

- Integer scaling with nearest-neighbour filtering as the default, and preserve the aspect ratio. Bilinear upscaling of pixel art looks bad and users will notice immediately, so this is not a place to accept a library default.
- Fullscreen, window resizing, and a sensible default size that is an integer multiple of the native resolution.

Deliverable: a clean, correctly scaled display.

Days 93–94 — The SP screen question

The one place the SP diverges — and it is worth doing properly.

The problem: artists colour-corrected their work for the original reflective, unlit, low-gamma panel. Raw emulator output on a modern monitor therefore looks harsh, oversaturated and too bright — not because your emulator is wrong, but because you are showing panel-uncorrected values on a panel they were never intended for. The SP's frontlit and backlit screens sit at different points on that same spectrum again, which is exactly why this component exists in an SP-targeted plan.

Offer three profiles: raw output, original-hardware correction, and SP-backlit correction. The correction is a channel-mixing matrix combined with a gamma adjustment; the SP profile uses a weaker mix and a lower gamma than the original-panel profile, reflecting the brighter, more saturated panel.

Precompute the entire correction into a lookup table over all possible fifteen-bit colours at startup rather than computing it per pixel. That is 32,768 entries computed once, after which the per-pixel cost is a single array read.

Consider also emulating the original panel's very slow pixel response as frame blending. Some games deliberately relied on it to produce transparency by alternating frames, and those effects look visibly wrong — flickering rather than blended — without it. This is a genuinely nice detail to be able to demonstrate side by side.

Deliverable: three selectable screen profiles driven by a lookup table, plus optional frame blending, with side-by-side comparison screenshots.

Day 95 — Quality-of-life features

- Save state slots with hotkeys, rewind implemented as a ring buffer of periodic states, pause, fast-forward, screenshots, a recent-ROM list, and drag-and-drop loading.
- None of these is technically interesting and all of them are expected. Their absence is what users notice, not their presence.

Deliverable: the standard feature set users expect from an emulator.

Day 96 — Command-line interface and headless modes

- Expose the headless runner, tracing and frame-limited execution as command-line options on the main binary, so that the debugging tooling from Component 4 is available to users and to CI without maintaining a separate build.

Deliverable: *a documented command-line interface covering normal, headless and trace modes.*

Day 97 — Buffer and usability test

- Hand it to someone who is not you and watch them open a ROM without any instructions. Every place they hesitate is a bug in the frontend, not a gap in their understanding.

Deliverable: *a stranger successfully loading and playing a game unaided.*

Component complete when: **someone else can use it without you in the room.**

Component 17 — Accuracy Hardening and Performance

Goal: close the accuracy debt you deliberately deferred, and then make it fast — in that order, because optimising an incorrect emulator means optimising work you are about to rewrite. **Duration:** 6 days.

Days 98–99 — The deferred accuracy list

Work the list you have been accumulating since Component 1, in descending order of how many programs actually care:

Item	Deferred from	Why it matters
Cartridge prefetch buffer	Component 1	The bus prefetches sequentially ahead of the processor, which materially changes ROM access timing. The highest-value remaining item by a clear margin.
Full open-bus behaviour	Component 1, Day 6	Pipeline-dependent and mode-dependent. A handful of games and several protection checks read it deliberately.
Per-scanline sprite cycle budget	Component 10, Day 61	Sprites beyond the budget are dropped. Most games stay under it; a few exceed it as an effect.
DMA and processor bus contention	Component 7	Affects timing when both compete for the same bus. Matters for tightly timed audio and raster code.
Picture-unit access restrictions	Component 8	Video and object memory are unavailable to the processor during specific rendering windows.
Horizontal blank flag at 1,006 cycles	Component 8, Day 47	A small correction with visible effects in a narrow set of raster-timed code.

- Some of these you will decide not to implement, and that is entirely fine. Write down which ones and why. A documented scope boundary reads as judgment; an undocumented one reads as an oversight, and the difference is visible to anyone reviewing the project.

Deliverable: the accuracy debt list resolved, each item either implemented or explicitly declined with written reasoning.

Day 100 — Profile before optimising

- Generate a flame graph from a headless run of several thousand frames. Expect the profile to be dominated by instruction dispatch, then memory access, then composition — but confirm it rather than assuming it.
- Resist optimising anything the profile does not implicate. This is the single most common way people waste a week on an emulator, and the fact that you can predict roughly what the profile will say is not a reason to skip generating it.

Deliverable: a flame graph and a written, ranked optimisation list derived from it.

Days 101–102 — The optimisations that pay

- Build settings first** — they are free and you configured most of them on Day 1. Full link-time optimisation, a single codegen unit, abort on panic, and native target CPU if you are not distributing binaries.

- **Bounds-check elimination in the memory path.** Use fixed-size arrays and mask indices so the compiler can prove the access is in range. Masking against a power-of-two size on a fixed-size array eliminates the check entirely; a modulo against a dynamically sized vector does not, and the difference is measurable in the hottest loop in the program.
- **Fast paths for the two regions instruction fetches actually come from,** checked before the general decode.
- **Const-generic instruction handlers,** if and only if the profile says dispatch is the bottleneck. Worth a meaningful fraction of core speed at real cost in boilerplate and compile time.
- **What not to bother with:** vectorising the compositor (too branchy, small win), writing a recompiler (a fascinating second project, not an optimisation — an interpreter already runs this machine many times faster than real time on modern hardware), and threading the core (the subsystems are too tightly coupled to separate cleanly).

***Deliverable:** measured speedup with before-and-after numbers from the same benchmark, not estimates.*

Day 103 — Buffer and compatibility sweep

- Run a broad set of games and record what works, what glitches, and what fails outright. Publish this honestly as a compatibility list.
- An accurate compatibility list that names its known failures is a stronger signal to a reviewer than a vague claim of broad support — it demonstrates that you tested systematically and that you know where the edges are.

***Deliverable:** a published compatibility list with specific, named known issues.*

Component complete when: the accuracy debt is resolved or documented, and headless throughput comfortably exceeds real time by a large margin.

Component 18 — Packaging and Release

Goal: turn five months of work into something someone can find, understand in five minutes, and respect in thirty. **Duration:** 5 days.

Day 104 — README as the front door

- A one-paragraph pitch, a screenshot, an architecture diagram, a quickstart, an explicit statement of the accuracy tier and BIOS strategy, and the compatibility list from Day 103.
- State plainly that this targets the SP and explain in one sentence what that means — identical hardware, different panel, hence the screen profiles. Pre-empting the obvious question is better than leaving it.
- Test the quickstart on a genuinely clean checkout, on a machine that has never built the project.

Deliverable: a README a stranger can follow to a running emulator.

Day 105 — Architecture writeup

- Assemble the deep-dive from the ADRs and component documents you kept throughout: the ownership model and why, the scheduler design, the accuracy tier and what it buys and costs, the scanline rendering choice and its limitation, and the audio synchronisation approach.
- This is assembly, not first-draft writing — but it only works if you actually wrote the documents as you went, which is why documentation appears in the deliverables of nearly every component in this plan rather than being deferred to the end.

Deliverable: an architecture deep-dive assembled from existing documents.

Day 106 — Demo and screenshots

- A short screen capture: boot, gameplay across two or three visually distinct games, the debugger showing layer toggles isolating a background, and a side-by-side of the three screen profiles.
- Show the test suite passing. For this kind of project a green suite of public hardware tests is more persuasive than any amount of gameplay footage, because it is the thing a reviewer cannot fake or misread.

Deliverable: a demo video and a set of clean screenshots.

Day 107 — Release engineering

- Tag a release with notes. Run a secrets scan across the full history. Build binaries for the platforms you can actually test on. Confirm the CI test-ROM suite runs green from a clean clone rather than from your working tree.

Deliverable: a tagged release with binaries and green CI from a clean clone.

Day 108 — Final buffer: done is done

- Use this day for anything still open. At the end of it, the project is finished.
- A completed, honestly scoped emulator that runs most games well beats a perpetually almost-finished one that would have been more accurate. Ship it.

Deliverable: ClamshellRS: complete, documented, released, and demo-ready.

Reference: Risk Register

Known friction points, worth re-reading before you reach each one.

Risk	Where it bites	Mitigation
Abandoning the project during the CPU	Components 2–3	Twenty working days with nothing visible produced. Pull the verification harness as early as it will go so you get advancing test counts rather than silence, and keep the Component 8 bitmap demo in sight as the near-term reward.
Proceeding with a failing processor	Component 3, Day 28	Treat this as a hard gate. A graphics bug sitting on a processor bug is ambiguous — you cannot tell whether the renderer is wrong or the value was wrong before it arrived — and untangling it costs days.
Debug-build performance	Every component	An unoptimised emulator runs at roughly two percent of target speed. Raise the development profile's optimisation level on Day 1, not the first time it becomes annoying.
The ownership model	Component 0, Day 2	Genuinely expensive to reverse once the CPU exists. Decide deliberately, and keep interior mutability out of the hot path or you will hit a re-entrant borrow panic during a DMA transfer in month three, with no obvious cause.
Barrel shifter edge cases	Component 2, Day 12	Zero encodings mean different things per operation, and register-supplied amounts change the rules again. Test the shifter exhaustively before any instruction depends on it, or one bug will present as fifteen.
Block transfer writeback	Component 2, Day 18	The single most common source of 'one specific game crashes at random'. Work the documented edge cases deliberately rather than discovering them from symptoms.
Interrupt acknowledgment inverted	Component 6, Day 38	Flags clear on writing a one, not a zero. Getting this backwards freezes games in a way that looks exactly like a processor fault.
Halt wake requiring the master enable	Component 6, Day 38	Halt wakes on an enabled pending interrupt regardless of the master enable. Requiring it hangs a subset of games permanently, with no symptom pointing at the cause.
Naive per-frame rendering	Component 8, Day 46	Breaks every raster effect, and games change display registers mid-frame constantly. Choose scanline rendering and record why.
Audio synchronisation deferred	Component 14, Days 84–86	Budget the full three days. It is consistently underestimated, and it is where audio quality actually comes from — sync to the audio device, never to a wall clock.
Optimising before profiling	Component 17, Day 100	Generate the flame graph first. The bottleneck is rarely exactly where intuition puts it, and a week spent optimising the wrong loop is a week gone.
BIOS and ROM distribution	Components 0 and 15	Never bundle a real BIOS dump or commercial cartridge dumps, and never commit them to the repository. Ship the open-source replacement; require user dumps and verify them by hash.
Scope creep into a recompiler	Component 17	A dynamic recompiler is a second project, not an optimisation. An interpreter already exceeds real-time speed on this hardware by a wide margin.
Frontend polish eating depth	Components 16 and 18	If time gets tight, cut frontend features before cutting accuracy work, testing, or documentation. The core is the portfolio value; the window is not.

Reference: Skills Map

Skill	Where it appears in ClamshellRS
Rust ownership and borrow modelling	Component 0's architecture decision; the machine struct with split borrows and the generic bus interface, applied throughout every later component
Systems programming	Memory mapping, address decoding, bit manipulation, fixed-point arithmetic, endianness, mirroring
Computer architecture	The full ARM7TDMI implementation: three-stage pipeline, seven processor modes, banked registers, exception vectoring
Instruction set implementation	Two complete instruction sets, compile-time dispatch table generation, condition evaluation, the barrel shifter
Discrete event simulation	The scheduler in Component 5: event queues, deterministic tie-breaking, halt fast-forwarding
Graphics pipelines	Tiled and bitmap rendering, affine transforms, scanline compositing, priority resolution, alpha blending, windowing
Digital audio	Waveform synthesis, sample queues, mixing, resampling, and host device synchronisation with dynamic rate control
Real-time systems	Frame pacing, audio-clocked synchronisation, latency management, the emulated-time versus real-time drift problem
Performance engineering	Profiling with flame graphs, bounds-check elimination, dispatch optimisation, measured before-and-after benchmarking
Reverse engineering and debugging	Trace diffing against a reference implementation, disassembly, differential debugging, ring-buffer crash dumps
Test engineering	Headless harnesses, ROM-based integration tests, framebuffer hashing for regression detection, determinism testing
Serialization	Save states with relative-time scheduler serialisation and round-trip determinism verification
Protocol implementation	Flash command sequences, serial EEPROM bit protocols, and optionally the standard remote debugging protocol
Technical writing and scoping	Architecture decision records, the deep-dive writeup, and an honest compatibility list with named known issues

Reference: Component-by-Component Timeline Summary

Component	Days	Length	Core output
0 — Environment and commitments	1–3	3 days	Workspace, CI, ownership model, accuracy tier, BIOS strategy
1 — Memory bus and cartridge	4–8	5 days	Full address decode, mirroring, BIOS protection, timing costs, ROM loading
2 — CPU: core and ARM set	9–20	12 days	Register banking, pipeline, barrel shifter, every ARM instruction
3 — CPU: THUMB and exceptions	21–28	8 days	Nineteen THUMB formats, exception mechanism, both test suites green
4 — Verification and tooling	29–33	5 days	Headless harness, trace diffing, disassembler, save states, debugger
5 — Timing and scheduler	34–37	4 days	Event queue, main loop ordering, halt fast-forwarding
6 — Interrupts and timers	38–41	4 days	Interrupt controller, four scheduled timers with cascade
7 — DMA	42–45	4 days	Four channels, priority and preemption, timing triggers, audio queue mode
8 — PPU: timing and bitmap modes	46–50	5 days	Video state machine, colour conversion, three bitmap modes, first image
9 — PPU: backgrounds	51–56	6 days	Text and affine backgrounds, layer toggles, first title screens
10 — PPU: sprites	57–62	6 days	128 sprites, affine transforms, priority, both mapping modes
11 — PPU: effects	63–69	7 days	Windows, blending, mosaic, the full compositor
12 — Input	70–71	2 days	Keypad, interrupts, bindings — the emulator becomes playable
13 — Backup memory	72–76	5 days	Type detection, static RAM, flash, EEPROM, atomic persistence
14 — Audio	77–88	12 days	Six channels, mixing, host synchronisation with dynamic rate control
15 — BIOS	89–91	3 days	Three boot paths, documented strategy, system-call reference
16 — Frontend and the SP screen	92–97	6 days	Presentation, three screen profiles, quality-of-life features
17 — Accuracy and performance	98–103	6 days	Deferred accuracy closed or declined, profiled optimisation, compatibility list
18 — Packaging and release	104–108	5 days	README, architecture writeup, demo, tagged release

Total: 108 working days at 4 hrs/day, 5 days/week — roughly 22 weeks, or 5 to 5.5 calendar months with normal life interruptions. Buffer days are already included within each component. Roughly a fifth of the total sits in Components 2 and 3, which is uncomfortable and unavoidable: the processor is the foundation, and every hour saved by rushing it is repaid with interest during the graphics work.

The schedule is a tool, not a contract. Hardware will occasionally force a detour the plan did not anticipate — an instruction edge case that is documented ambiguously, a game that depends on behaviour nobody wrote down, a timing interaction that only shows up in one sound engine. Those detours are the actual substance of emulator development, not setbacks from it.

The parts worth protecting if time gets tight are consistent throughout: **processor correctness, the verification harness, honest scope documentation, and audio synchronisation**. Everything else — frontend polish, the accuracy long tail, the debugger's nicer views — is negotiable. Those four are not.

Reference: Symptom-to-Cause Triage Table

When something breaks and you have no idea why, work down this list before doing anything else. It covers the large majority of bugs this project will produce.

Symptom	Look first at	Component
Game hangs immediately at boot	Branch-and-exchange, mode switching, THUMB entry	2, 3
Game hangs after a few frames	Interrupt return: stored link-register offsets, and the status restore on a flag-setting instruction with the program counter as destination	2, 3
An interrupt handler fires forever	Request flags being cleared by writing zero rather than one	6
Halt never wakes	The wake condition incorrectly requiring the master enable	6
Garbled graphics, correct layout	Palette handling, or four-bit nibble ordering within tile bytes	9
Correct graphics, wrong position	Scroll offset handling, or affine reference latching	9
Left half correct, right half garbled	Wide tilemap screen block index computation	9
Sprites missing entirely	Sprite video RAM base, or the bitmap-mode tile index restriction	10
Sprites flickering	Priority tie-breaking by object memory index	10
Sprites teleporting near screen edges	Nine-bit signed horizontal or eight-bit wrapping vertical position	10
Effects render as solid blocks	Semi-transparent sprite override, or the immediately-below rule for the second blend target	11
Screen uniformly too dark or too bright	Blend target selection, or the window bit that gates blending	11
Audio crackling	Buffer underrun — you are syncing to a wall clock rather than the audio device	14
Audio pitch or tempo wrong	Timer handling for the queue clock timers	6, 14
Saves do not persist	Save-type detection; check the game-code override table	13
Random crashes in one specific game	Block transfer edge cases, or unaligned load rotation	2
Everything correct but slow	A debug build, or per-cycle ticking instead of the scheduler	0, 5
Determinism test fails intermittently	Scheduler tie-breaking not being deterministic by insertion order	5

When none of the above fits

Trace diff against your reference emulator. Run both against the same ROM from the same starting state, diff the register dumps, and find the first line where they disagree. The instruction immediately before that line is almost always the bug.

This works every time, it works on bugs you have no theory about, and it is the reason Component 4 exists at the position it does in this plan rather than at the end.